

Laboratorio: Registro binario de estudiantes

C++20 — archivo `estudiante.bin`

1. Formato del archivo

Cada estudiante debe almacenarse campo por campo, en este orden exacto:

```
int codigo;
unsigned nombreLen;
char nombre[nombreLen];
unsigned carreraLen;
char carrera[carreraLen];
double promedio;
int clasesAprobadas;
```

Formato físico:

```
[codigo]
[nombreLen]
[nombre bytes...]
[carreraLen]
[carrera bytes...]
[promedio]
[clasesAprobadas]
```

Los strings no incluyen `\0`. No escribas el objeto `Estudiante` completo con `sizeof(Estudiante)` si contiene `std::string`.

2. Modelo en memoria

```
struct Estudiante {
    int codigo;
    std::string nombre;
    std::string carrera;
    double promedio;
    int clasesAprobadas;
};
```

3. agregarEstudiante

```
bool agregarEstudiante(
    const std::string& path,
    int codigo,
    const std::string& nombre,
    const std::string& carrera,
    double promedio,
    int clasesAprobadas);
```

Validar:

- `codigo > 0`
- `nombre` no vacío

- `carrera` no vacía
- `0 <= promedio <= 100`
- `clasesAprobadas >= 0`
- El código no puede repetirse.

Antes de agregar, recorre el archivo y comprueba si el código ya existe. Si todo es válido, agrega el estudiante al final y retorna `true`; si no, retorna `false` sin modificar el archivo.

4. listarEstudiantes

```
void listarEstudiantes(const std::string& path);
```

Recorre secuencialmente todo `estudiante.bin`. Reconstruye los strings leyendo primero su longitud y luego exactamente esa cantidad de bytes.

Por cada estudiante muestra código, nombre, carrera, promedio y clases aprobadas.

No uses `while (!file.eof())`. Distingue entre final normal del archivo y registro incompleto.

5. buscarEstudiante

```
bool buscarEstudiante(
    const std::string& path,
    int codigo,
    Estudiante& resultado);
```

Debe abrir el archivo, recorrer registros, encontrar el código solicitado, reconstruir completamente el estudiante, asignarlo a `resultado` y retornar `true`. Si no existe, retorna `false`.

6. actualizarPromedio

```
bool actualizarPromedio(
    const std::string& path,
    int codigo,
    double nuevoPromedio);
```

Validar `0 <= nuevoPromedio <= 100`. Recorre el archivo hasta encontrar el estudiante y determina durante el recorrido la posición exacta donde comienza `double promedio`. Sobrescribe únicamente ese `double` mediante posicionamiento y `write()`.

7. aprobarClase

```
bool aprobarClase(
    const std::string& path,
    int codigo);
```

Busca el estudiante, incrementa `clasesAprobadas` y modifica únicamente ese `int` dentro del archivo.

8. mejorEstudiante

```
bool mejorEstudiante(  
    const std::string& path,  
    Estudiante& resultado);
```

Recorre todo el archivo y obtiene el estudiante con mayor promedio. Si el archivo está vacío, retorna `false`; si existe al menos uno, almacena el estudiante en `resultado` y retorna `true`.

9. Auxiliares

Puedes diseñar funciones auxiliares equivalentes a:

```
escribirString(...)  
leerString(...)  
leerEstudiante(...)
```

10. Estado del stream

Si una lectura llevó el stream a EOF/fallo y después necesitas reposicionarlo, usa `file.clear()` antes de `seekg()` o `seekp()` cuando corresponda.

11. Secuencia de prueba

Prueba, como mínimo, esta secuencia:

1. Agregar estudiante 1001.
2. Agregar estudiante 1002.
3. Agregar estudiante 1003.
4. Intentar duplicar 1001.
5. Listar.
6. Buscar 1002.
7. Modificar promedio de 1002.
8. Aprobar una clase de 1001.
9. Listar nuevamente.
10. Mostrar mejor estudiante.
11. Cerrar y volver a ejecutar para verificar persistencia.

Datos sugeridos:

```
1001 | Ana Lopez | Ingenieria en Sistemas | 84.50 | 19  
1002 | Carlos Mendez | Ingenieria Industrial | 91.75 | 27  
1003 | Lucia Torres | Ingenieria en Sistemas | 88.20 | 31
```

Después intenta agregar nuevamente el código 1001 con datos distintos; debe rechazarse.